Homography Synchronization over Graphs: Iterative, Spectral, and Robust Methods

May 27, 2026

Abstract

This report describes the design, implementation, and experimental evaluation of a framework for synchronizing 3×3 projective transformations (homographies) over a graph of pairwise relative measurements. Two main families of algorithms are implemented: *iterative* methods based on Madhavan, Fusiello, and Arrigoni [1], offering Euclidean, Direction, and Sphere (L1 geodesic mean) averaging with optional IRLS outlier suppression; and *spectral* (closed-form) methods based on Schroeder, Bartoli, Georgel, and Navab [2], providing Locally Scaled Homographies (LSH) and Globally Scaled Homographies (GSH). A spanning-tree baseline is included for reference. The framework is benchmarked on synthetic graphs—varying graph size, noise level, edge density, outlier ratio, and topology—as well as on real images via a full SIFT-based mosaicking pipeline. Results demonstrate that Sphere averaging is the most accurate iterative method overall, that IRLS variants dramatically outperform plain methods in the presence of outliers, and that the choice of graph topology significantly affects relative algorithm performance.

1 Introduction

Given a set of n views (images), the *homography synchronization* problem seeks to recover the unknown absolute projective transformations $\{X_i\}_{i=1}^n$, $X_i \in \mathbb{R}^{3\times3}$, from a set of noisy pairwise measurements $Z_{ij} \approx X_i X_j^{-1}$ available on the edges of a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. This problem arises naturally in image mosaicking, Structure-from-Motion, and multi-view geometry, wherever relative inter-frame transformations are estimated independently and must be made globally consistent.

The synchronization solution is inherently defined up to a global right multiplication: if $\{X_i\}$ is a solution, so is $\{X_i \cdot C\}$ for any invertible C. All methods in this project operate in SL(3)—the group of 3×3 real matrices with unit determinant—which removes the scale ambiguity inherent to projective transformations and turns the approximate consistency relation $Z_{ij} \simeq X_i X_j^{-1}$ into the strict equality $Z_{ij} = X_i X_j^{-1}$.

2 Mathematical Formulation

2.1 The Synchronization Graph

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph where each vertex $i \in \mathcal{V}$ carries an unknown absolute homography $X_i \in \mathrm{SL}(3)$, and each edge $(i, j) \in \mathcal{E}$ carries a measured relative homography Z_{ij}. In the noise-free case:

$$Z_{ij} = X_i \, X_j^{-1}, \qquad \forall\, (i,j) \in \mathcal{E}. \tag{1}$$

Both directions are stored: $Z_{ji} = Z_{ij}^{-1}$.

2.2 SL(3) Normalisation

Every 3×3 matrix H is projected into SL(3) by dividing by the cube root of its determinant:

$$\bar{H} = \frac{H}{\sqrt[3]{\det(H)}}, \qquad \det(\bar{H}) = 1. \tag{2}$$

For 3×3 real matrices the cube root is always real, making this normalisation straightforward.

2.3 Error Metric

Since the solution is defined up to a global transformation C, evaluation first aligns the estimate to the ground truth by solving $C = \hat{X}_0^{-1} X_0$ using vertex 0 as anchor. The angular error for vertex i is then:

$$e_i = \arccos\Big(\big| \operatorname{vec}(\hat{X}_i C)^\top \operatorname{vec}(X_i) \big| \Big), \tag{3}$$

where $\operatorname{vec}(\cdot)$ denotes unit-norm flattening to $\mathbb{R}^9$. The absolute value handles the antipodal equivalence on the sphere. The final metric is the mean angular error in degrees across all vertices.

3 Synchronization Algorithms

3.1 Iterative Methods (Madhavan et al. [1])

Each vertex i is updated in turn by computing *neighbour estimates*

$$X_{i|j} = Z_{ij}\, X_j, \qquad \forall\, j \in \mathcal{N}(i), \tag{4}$$

and then averaging these estimates to obtain a new X_i. Vertices are processed in order of descending degree, since high-degree nodes receive more constraints and are therefore more stable. The sweep repeats until the maximum entry-wise change drops below a tolerance τ.

Three notions of average are implemented:

Euclidean averaging. Normalise each estimate to unit Frobenius norm, compute the weighted componentwise mean, then re-normalise:

$$c = \frac{\sum_k w_k\, \bar{h}_k}{\| \sum_k w_k\, \bar{h}_k \|}, \tag{5}$$

where $\bar{h}_k = \operatorname{vec}(X_{i|j_k}) / \| \operatorname{vec}(X_{i|j_k}) \|$.

Direction averaging. Finds the eigenvector of $M = \sum_k w_k\, (h_k h_k^\top)/(h_k^\top h_k)$ corresponding to the largest eigenvalue. This is L2, scale-invariant.

Sphere averaging (L1 geodesic mean). Minimises $\sum_k w_k \arccos(c^\top \bar{h}_k)$ subject to $\|c\| = 1$ via the fixed-point iteration:

$$c \leftarrow \alpha \sum_k \frac{w_k\, \bar{h}_k}{\sqrt{1 - (c^\top \bar{h}_k)^2}}, \tag{6}$$

iterated until convergence (≤ 50 steps, or $\|c_{\text{new}} - c\|_\infty < 10^{-8}$).

3.1.1 IRLS Outlier Suppression

When outlier edges are present, an outer *Iteratively Reweighted Least Squares* (IRLS) loop wraps the inner synchronization sweep. After each full convergence, per-edge Cauchy weights are recomputed from the residual:

$$w_{ij} = \frac{1}{1 + (r_{ij}/s)^2}, \tag{7}$$

where r_{ij} is the angular residual (in radians) between $Z_{ij}X_j$ and X_i, and s is the Cauchy scale parameter (typically 0.05–0.3). Inlier edges receive weight ≈ 1, while gross outliers are suppressed toward 0.

The inner sweep then restarts with updated weights. This procedure is repeated for a fixed number of IRLS iterations (typically 5), requiring no explicit outlier detection.

3.2 Spectral Methods (Schroeder et al. [2])

These methods build a $3n \times 3n$ block matrix from all known inter-frame homographies and extract the absolute homographies from its null space via SVD.

LSH (Locally Scaled Homographies). Builds the block matrix S with entries:

$$S[k, i] = \frac{\gamma_{i,k}}{\zeta_k} H_{i,k}, \tag{8}$$

where $\gamma_{i,k} = 1$ if the homography from i to k is known, and $\zeta_k = \sum_i \gamma_{i,k}$ counts known connections to node k (including self). In the noise-free case $S \cdot U = U$, so U lies in the null space of $(S - I)$. The three right singular vectors of $(S - I)$ with the smallest singular values approximate U.

GSH (Globally Scaled Homographies). Corrects the weighting bias of LSH by building:

$$G[k, k] = (1 - \zeta_k)\, I, \qquad G[k, i] = \gamma_{i,k}\, H_{i,k} \quad (i \neq k). \tag{9}$$

The absolute homographies are again extracted from the null space of G via SVD.

Both methods handle missing data naturally (not all pairs need to be connected). Their cost is $O((3n)^3)$ in the SVD, making them fast for moderate graph sizes but less scalable than iterative methods for very large graphs.

3.3 Spanning-Tree Baseline

The simplest approach: find a BFS spanning tree rooted at the highest-degree vertex and chain relative homographies along each path:

$$X_{\text{child}} = Z_{\text{child,parent}} \cdot X_{\text{parent}}. \tag{10}$$

This is $O(n)$ and very fast, but accumulates errors from root to leaves and does not exploit redundant measurements.

4 Image Mosaicking Pipeline

The `mosaic.py` module implements a full image stitching pipeline that applies homography synchronization to real images:

1. **Feature extraction:** SIFT keypoints and 128-dimensional descriptors are computed on CPU using OpenCV's `SIFT_create()`.

2. **Descriptor matching:** A GPU-accelerated brute-force matcher (`cv.cuda.DescriptorMatcher`) with L2 norm is used when CUDA is available; otherwise a CPU `BFMatcher` fallback is employed. Features are extracted once per image—$O(N)$—rather than re-extracted inside the $O(N^2)$ matching loop.

3. **Pairwise homography estimation:** Lowe's ratio test (threshold 0.7) filters ambiguous matches from repetitive patterns. RANSAC (inlier tolerance 5.0 px, minimum 10 inliers) robustly estimates each pairwise homography.

4. **Graph construction:** Vertices are images; each successfully matched pair becomes an edge carrying its relative homography Z_{ij}.

5. **Synchronization:** Any of the methods from Section 3 is applied.

6. **Mosaic rendering:** Each image is warped into the reference frame using the synchronised absolute homographies, via GPU-accelerated `warpPerspective` when available. A translation matrix ensures all coordinates are non-negative.

Quality is evaluated using *mean reprojection error*: for every matched edge (i, j), inlier points from image i are warped into image j using the synchronised transform $\hat{Z}_{ji} = \hat{X}_j \hat{X}_i^{-1}$, and the pixel distance to the actual matched keypoint is measured. Lower values indicate better consistency; typical good values are < 5 px.

5 Source Code Overview

The `src/` directory contains the following modules:

`graph.py` **(1466 lines)**
: Core module implementing the `Graph` class with adjacency structure, SL(3) normalisation, all three iterative averaging functions (Euclidean, Direction, Sphere), IRLS weight computation with Cauchy robust function, the spectral synchronization methods (LSH, GSH), and the spanning-tree baseline. Also provides utilities for synthetic graph construction (random Erdős–Rényi, linear chain, circular ring, 2-D grid, and multi-lane topologies), ground-truth generation in SL(3), noise injection, outlier edge insertion, angular error computation, and graph visualization via NetworkX and Matplotlib.

`mosaic.py` **(601 lines)**
: Full image mosaicking pipeline: SIFT feature extraction, GPU/CPU BFMatcher descriptor matching, Lowe's ratio test, RANSAC homography estimation, graph construction from images, reprojection error computation, mosaic rendering with GPU-accelerated warp, and command-line interface with configurable synchronization method, averaging variant, IRLS iterations, and Cauchy scale.

`benchmark.py` **(831 lines)**
: Main benchmarking script running five experiments (Sections 7.1–7.5). Implements the method registry (6 plain methods + 3 IRLS variants), a single-trial runner supporting all topologies, experiment sweep functions, consistent plotting with per-method visual styles, a combined results tiler, and a command-line `-from N` option for resuming from a specific experiment.

`benchmark_noise_topology.py` **(549 lines)**
: Joint noise × topology benchmark (Section 7.6). Sweeps noise levels across six graph topologies (linear bw=1, linear bw=3, circular chord=3, grid 8-connected, multilane 3 lanes, random E-R) and produces a grid of error-vs-noise line plots, a per-method topology-comparison plot, and a winner heatmap.

`compress.py` **(49 lines)**

Utility for resizing and JPEG-compressing a dataset of images to accelerate processing and reduce memory usage during mosaicking (default: 25% scale, quality 100).

6 Experimental Setup

All synthetic experiments use 50 independent trials per data point, and the reported metric is the *median* angular error (in degrees) to reduce sensitivity to rare divergence events. Unless otherwise noted, the default graph topology is random (Erdős–Rényi with edge dropout), with $n = 25$ nodes.

Six synchronization methods are compared in Experiments 1–3: **Tree** (baseline), **Sphere**, **Euclidean**, **Direction** (iterative), **LSH**, and **GSH** (spectral). Experiment 4 adds three IRLS variants: **Sphere-IRLS**, **Euclidean-IRLS**, and **Direction-IRLS** (5 IRLS iterations, Cauchy scale $s = 0.15$).

7 Experimental Results

7.1 Experiment 1: Varying Graph Size

Setup: $n \in \{10, 20, 30, 50, 75, 100\}$, $\sigma = 0.05$, $\rho = 0.5$ (hole density), $\gamma = 0$ (no outliers).

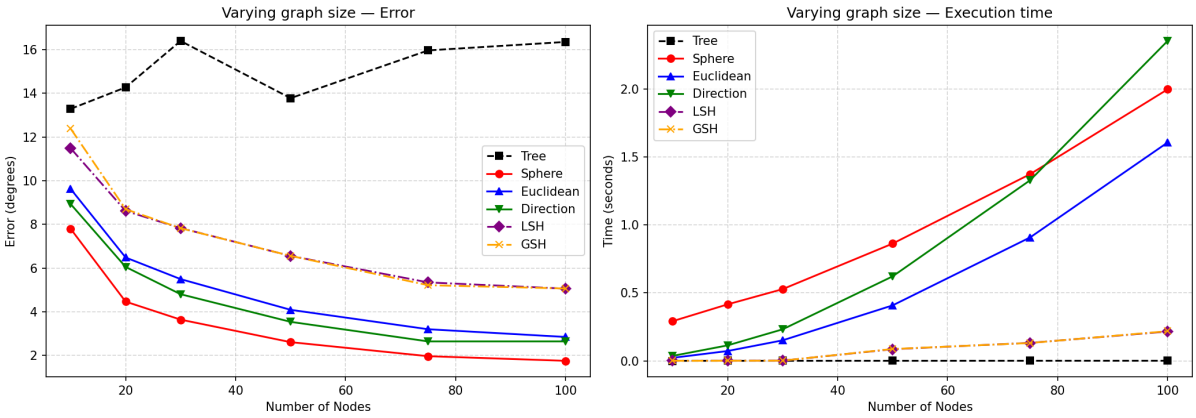

Figure 1: Experiment 1: Angular error (left) and execution time (right) as a function of graph size.

Observations: As the graph grows, more redundant measurements become available, and all iterative/spectral methods improve—Sphere averaging consistently achieves the lowest error (from $\sim 8°$ at n=10 down to $\sim 2°$ at n=100). The spanning-tree baseline *worsens* with size (from $\sim 13°$ to $\sim 16°$), confirming its error accumulation from root to leaves. On execution time, LSH/GSH remain the fastest synchronization methods (single SVD solve), while Sphere is the slowest due to its per-node fixed-point iteration; the tree baseline is near-instant.

7.2 Experiment 2: Varying Noise Level

Setup: $\sigma \in \{0.01, 0.02, 0.04, 0.06, 0.08, 0.10, 0.12, 0.15\}$, $n = 25$, $\rho = 0.5$, $\gamma = 0$.

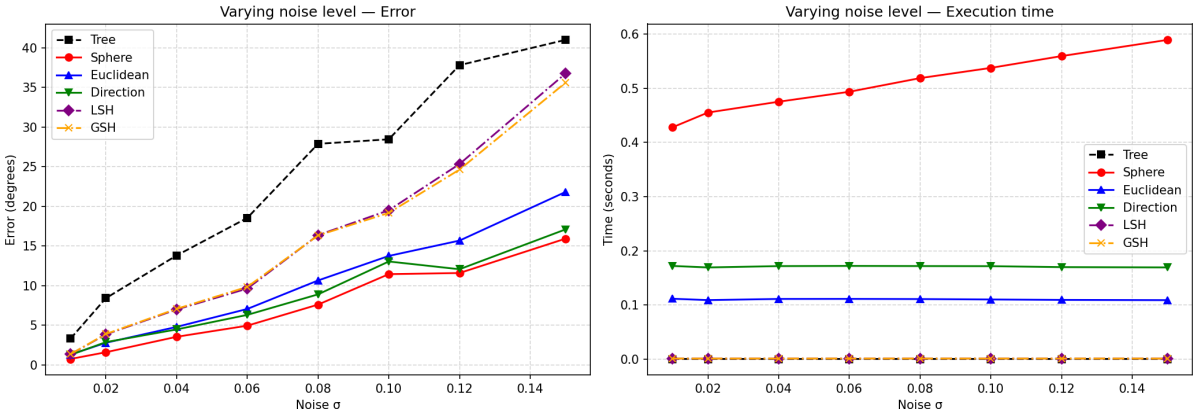

Figure 2: Experiment 2: Angular error (left) and execution time (right) as noise increases.

Observations: All methods degrade with increasing noise, but the iterative and spectral approaches degrade more gracefully than the spanning tree. Sphere averaging maintains the lowest error throughout (from $\sim$1° at σ=0.01 to $\sim$16° at σ=0.15), while LSH/GSH degrade more rapidly and converge toward the tree baseline at high noise. Execution times are largely independent of noise level.

7.3 Experiment 3: Varying Hole Density

Setup: $\rho \in \{0.0, 0.1, 0.2, 0.3, 0.5, 0.7, 0.8, 0.9\}$, $n = 25$, $\sigma = 0.05$, $\gamma = 0$.

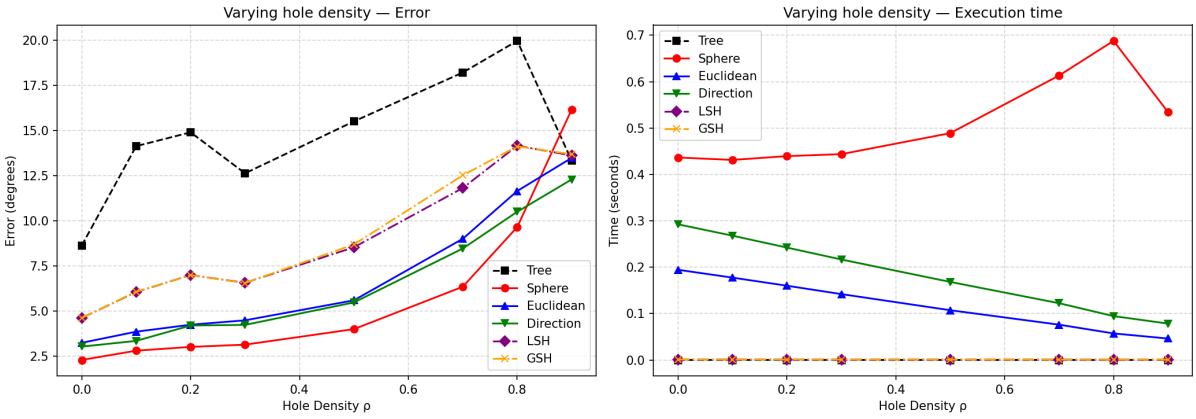

Figure 3: Experiment 3: Angular error (left) and execution time (right) as the fraction of missing edges increases.

Observations: With denser graphs ($\rho \to 0$), all methods benefit from abundant redundancy. As more edges are removed, less redundancy is available for error compensation and all methods converge toward spanning-tree performance. Sphere maintains the best accuracy across the full range; the spectral methods (LSH/GSH) handle missing data natively and remain competitive until very high sparsity ($\rho > 0.7$). Iterative execution time decreases with sparsity (fewer neighbours to average), while spectral time is roughly constant (dominated by the fixed $3n \times 3n$ SVD).

7.4 Experiment 4: Varying Outlier Density

Setup: $\gamma \in \{0.0, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6\}$, $n = 25$, $\sigma = 0.05$, $\rho = 0.2$.

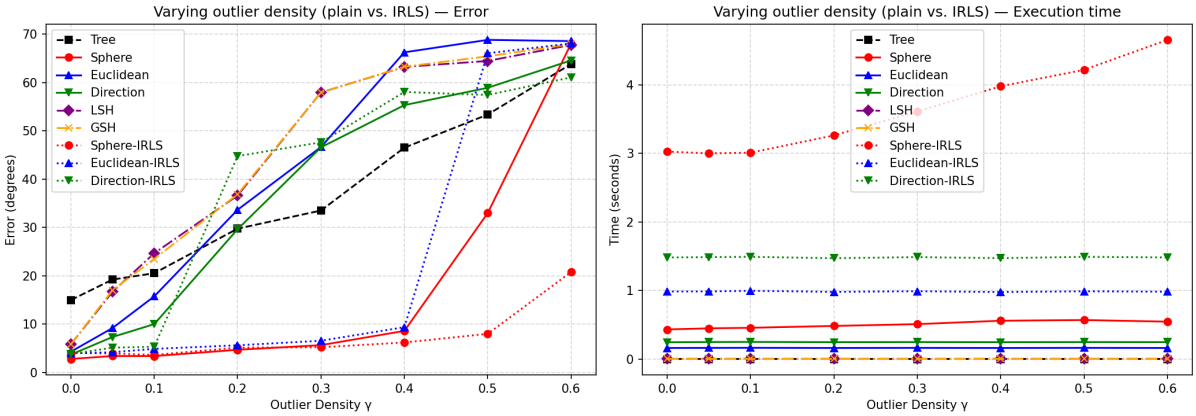

Figure 4: Experiment 4: Plain methods (solid) vs. IRLS variants (dotted). Left: angular error; right: execution time.

Observations: This experiment clearly demonstrates the value of IRLS robust reweighting. All plain methods degrade rapidly beyond γ=0.1, reaching ∼60–70° error at γ=0.6—effectively random. In contrast, the **Sphere-IRLS** variant maintains ∼5–8° error even at γ=0.5, only degrading significantly at γ=0.6. Euclidean-IRLS is the second-best robust method. The spectral methods (LSH/GSH) are not inherently robust and deteriorate as rapidly as plain iterative methods. The IRLS cost is modest: Sphere-IRLS takes ∼4 s at γ=0.6 compared to ∼0.5 s for plain Sphere.

7.5 Experiment 5: Real Image Data

Setup: 10 images from a compressed dataset, evaluated by mean reprojection error (pixels).

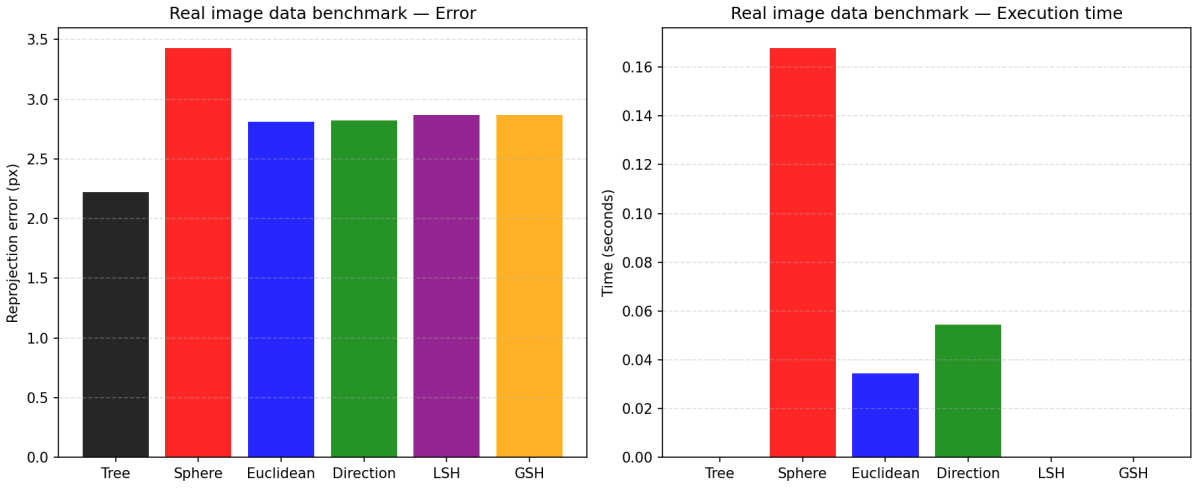

Figure 5: Experiment 5: Reprojection error (left) and execution time (right) on real images.

Observations: On real data, the **Tree** baseline achieves the lowest reprojection error (∼2.2 px), while the iterative and spectral methods cluster around 2.8–3.4 px. This counter-intuitive result arises because the real dataset has a predominantly linear (sequential) capture geometry with low noise, where the spanning tree already provides an accurate solution; the iterative/spectral methods may introduce slight over-correction. Sphere averaging is the slowest (∼0.16 s) while LSH and GSH are near-instantaneous.

7.6 Experiment 6: Joint Noise × Topology Analysis

Setup: $\sigma \in \{0.02, 0.04, 0.10, 0.15, 0.20, 0.25\}$, $n = 25$, $\rho = 0.3$, 20 trials, across six topologies: Linear (bw=1), Linear (bw=3), Circular (chord=3), Grid (8-connected), Multilane (3 lanes), Random (E-R).

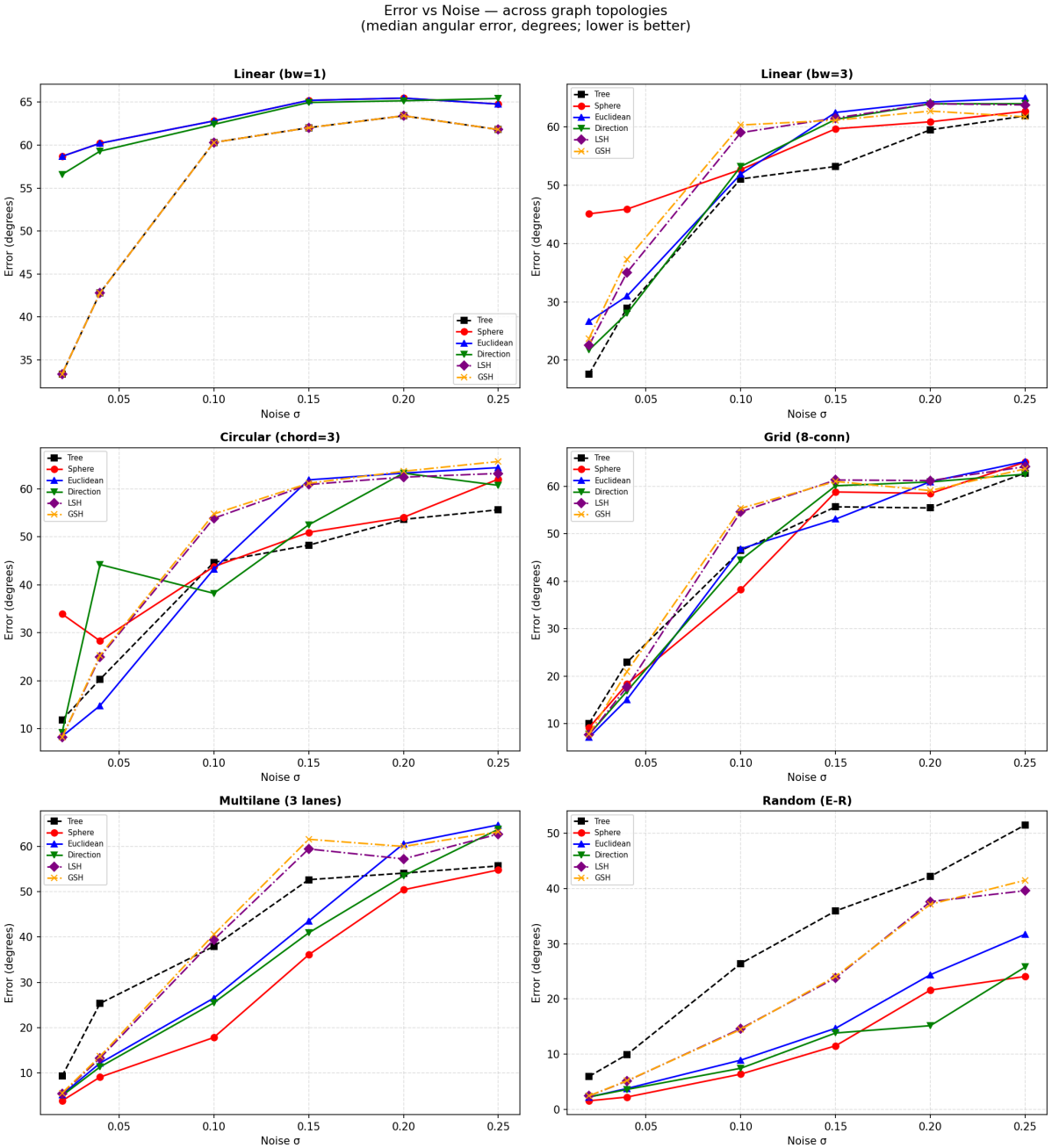

Figure 6: Error vs. noise across six graph topologies. Each subplot shows all methods on one topology; lower is better.

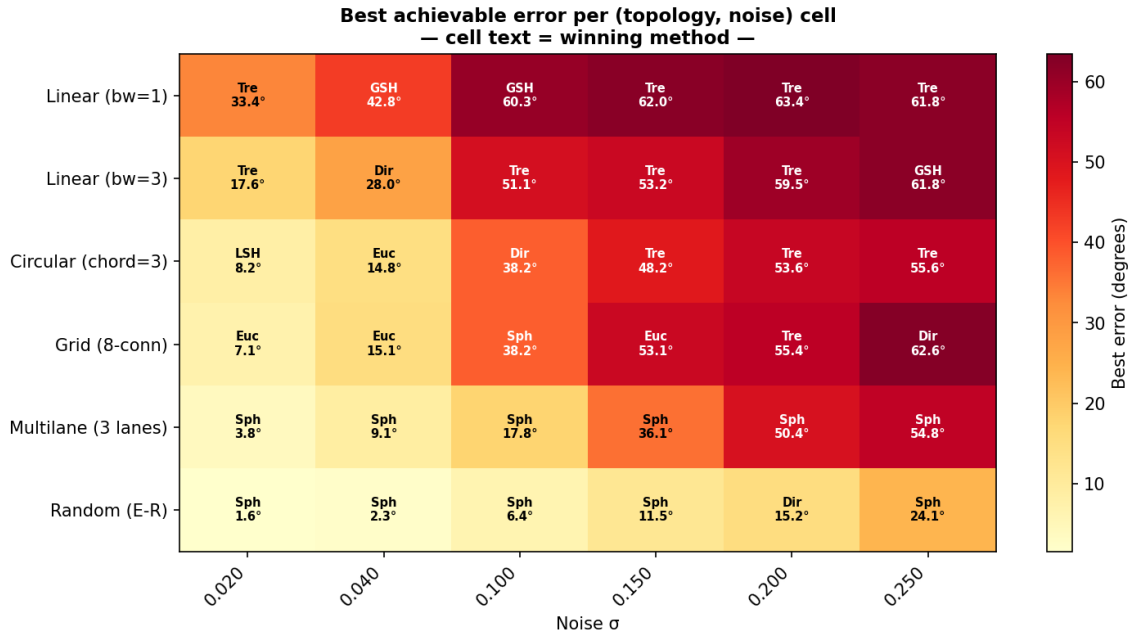

Figure 7: Winner heatmap: best method at each (topology, noise) cell. Cell colour encodes best achievable error; text shows the winning method abbreviation and error.

Observations from the grid plot (Figure 6):

- **Linear (bw=1):** All methods perform poorly due to minimal redundancy (pure chain). Tree and GSH are competitive at low noise; all methods saturate near 60–65° at high noise.
- **Linear (bw=3):** The wider bandwidth adds shortcut edges, improving all methods. Tree wins at low noise; Sphere and Direction recover at moderate noise.
- **Circular (chord=3):** Loop closure helps all methods. LSH and Euclidean excel at low noise; Direction and Sphere perform well at moderate noise.
- **Grid (8-connected):** Dense local redundancy benefits spectral and iterative methods equally. Euclidean excels at low noise; Sphere at moderate noise.
- **Multilane (3 lanes):** Sphere dominates across all noise levels, consistent with this topology being a realistic proxy for multi-strip aerial survey.
- **Random (E-R):** With high average connectivity, Sphere achieves the best performance overall, followed by Direction.

Observations from the heatmap (Figure 7): No single method dominates all conditions. **Sphere** wins most cells on random, multilane, and grid topologies. **Tree** is competitive on linear graphs, especially at moderate-to-high noise where iterative methods suffer from the lack of redundancy. Spectral methods (GSH, LSH) win only on sparse topologies at low noise. **Euclidean** is strong on grid and circular topologies at low noise.

8 Summary of Findings

Table 1: Comparative summary of synchronization methods.

Family	Strengths	Weaknesses
Iterative (Sphere)	Best accuracy overall; inherent L1 robustness; handles large graphs efficiently ($O(n \cdot d)$ per iteration)	Requires multiple iterations to converge; slowest per-node computation due to fixed-point loop
Iterative (Euclidean)	Competitive accuracy; simpler than Sphere; strong on grid/circular topologies	L2 averaging; less robust than Sphere
Iterative (Direction)	Scale-invariant; good on dense graphs	L2-based eigenvector; less robust to outliers
IRLS variants	Dramatic improvement under outliers ($\gamma \leq 0.5$); Cauchy weighting suppresses gross errors without explicit detection	5× overhead from repeated convergence cycles
Spectral (LSH/GSH)	Single SVD solve—no iterations; handles missing data natively; fastest synchronization step	$O((3n)^3)$ SVD cost; not inherently robust to outliers; less accurate than iterative on dense graphs
Spanning Tree	Fastest method (single BFS, $O(n)$); works well on low-noise linear data	Accumulates errors from root to leaves; does not exploit redundant measurements

Key takeaways:

1. **Sphere averaging is the best general-purpose method**, with the lowest error on random and multilane topologies across all noise levels.
2. **IRLS is essential for outlier-contaminated data**: Sphere-IRLS maintains $< 10°$ error even with 50% outlier edges.
3. **Topology matters**: linear chains with low bandwidth severely limit all methods due to minimal redundancy; adding loop closure (circular) or cross-connections (multilane, grid) dramatically improves performance.
4. **Spectral methods are efficient but fragile**: LSH/GSH excel on clean, moderately sparse graphs but degrade rapidly with noise or outliers.
5. **The spanning tree can be surprisingly competitive** on sequential (linear) real-world datasets with low noise.

References

[1] R. Madhavan, A. Fusiello, and F. Arrigoni, "Synchronization of Projective Transformations," 2024.

[2] P. Schroeder, A. Bartoli, P. Georgel, and N. Navab, "Closed-Form Solutions to Multiple-View Homography Estimation," in *Proc. IEEE Workshop on Applications of Computer Vision (WACV)*, 2011.